API Authentication: API Keys vs JWT vs HMAC vs mTLS

five ways to prove who is calling, and where each fits

Watch the 4-min video

> **In one line:** API keys identify an app but are long-lived bearer secrets; session cookies suit browsers; bearer tokens (JWT or opaque) suit users and services; HMAC signing proves the sender and the body without sending the secret; mutual TLS proves service identity at the connection level.

1 Different callers, different proofs

A mobile app, a partner's backend, a webhook from a payment provider and an internal service all call your API. Each one has to prove who it is, but no single method fits all of them. There are five common methods: a browser or mobile app usually logs a user in (session cookie or OAuth token), a partner backend uses OAuth client credentials or HMAC, a webhook carries an HMAC signature, and an internal service uses mutual TLS.

2 API keys and session cookies

API keys are the simplest: a long random string sent in a header, such as `Authorization: Bearer sk_live_…` or `X-API-Key`. A key identifies the **calling application**, which is great for usage tracking and rate limits. But it is a **bearer secret**: anyone who sees it can use it, it rarely expires, and it doesn't identify a user. So:

- **Store only a hash** on the server (like a password), so a database leak doesn't expose usable keys. Show the full key to the customer once.
- **Support rotation**: allow two active keys at once, so a customer can switch without downtime.
- **Never ship a key inside a mobile app** or front-end code. Anyone can extract it from the app package.
- Send it in a header, never in the URL, where it ends up in logs and browser history.

Session cookies are the classic for browsers. After login, the server stores a session and sends a cookie holding a random session ID. Mark it `HttpOnly` (scripts can't read it, which blunts XSS), `Secure` (HTTPS only) and `SameSite` (other sites can't send it, which blunts CSRF). Sessions are easy to revoke (delete the row), but they need a session store, and cookies don't suit server-to-server calls.

3 Bearer tokens: JWT vs opaque

Bearer tokens, usually issued by an **OAuth 2** authorization server, travel in the `Authorization: Bearer …` header. There are two kinds:

- A **JWT** is self-contained and signed. Any service can verify it locally with the issuer's public key, with no database lookup. But it is hard to revoke before it expires, so keep lifetimes short, like **15 minutes**, and use refresh tokens to get new ones.
- An **opaque token** is just a random reference. The server must look it up (or call the auth server's introspection endpoint), but revocation is instant.

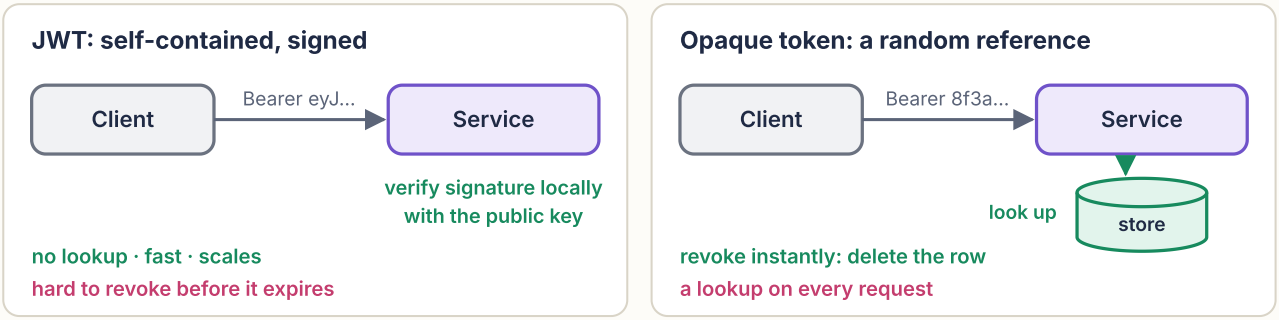


Figure 1. The core trade-off: JWTs save a lookup on every request; opaque tokens make revocation immediate.

For server-to-server calls with no user, OAuth 2 has the **client credentials** flow: the service authenticates with its client ID and secret and gets a short-lived access token. That beats a long-lived static key, because a leaked token stops working within minutes.

4 HMAC request signing

HMAC signing goes a step further. Client and server share a secret, but **the secret is never sent**. The client computes an **HMAC-SHA256** over the method, path, timestamp and body, and sends only the signature. The server recomputes it with its copy of the secret and compares.

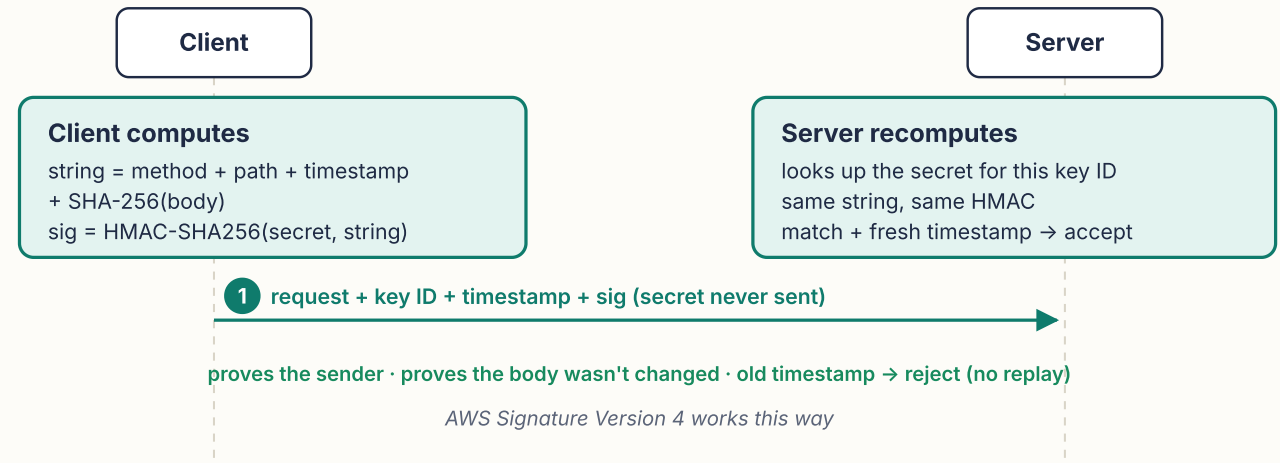


Figure 2. HMAC request signing. Changing a single byte of the body or path produces a completely different signature.

That proves **who sent the request** and that **nobody changed it in transit**. Rejecting old timestamps blocks **replay attacks**, where an attacker resends a captured request. AWS Signature Version 4 works this way (it also signs selected headers and derives a per-day, per-region signing key from the secret).

WORKED EXAMPLE: A WEBHOOK RECEIVER

A payment provider posts a `payment.succeeded` event to your endpoint with a signature header. To accept it safely:

1. Read the **raw body**, before any JSON parsing. Re-serialized JSON may differ by a single space, and the signature won't match.
2. Compute the HMAC with your webhook secret, over the timestamp and the body.
3. Compare using a **constant-time comparison**, to avoid timing attacks (a normal `==` stops at the first wrong byte, which leaks how much matched).
4. Reject timestamps older than **5 minutes**.

5. Only then process the event, **idempotently**, by its event ID: providers retry, so the same event can arrive twice.

```
import hmac, hashlib, time

def verify_webhook(raw_body: bytes, ts: str, sig: str, secret: bytes) -> bool:
    if abs(time.time() - int(ts)) > 300:          # older than 5 minutes: possible replay
        return False
    signed = ts.encode() + b"." + raw_body          # exact bytes, BEFORE any JSON parsing
    expected = hmac.new(secret, signed, hashlib.sha256).hexdigest()
    return hmac.compare_digest(expected, sig)       # constant-time comparison

# then: if event_id already processed -> return 200 and skip (idempotent)
```

5 Mutual TLS

In normal TLS only the server shows a certificate. In **mutual TLS (mTLS)**, both sides present certificates during the handshake, so the server knows exactly which service is calling **before reading a single byte of HTTP**. It is the strongest option for service-to-service traffic, and service meshes like **Istio** or **Linkerd** issue and rotate short-lived certificates automatically (by default they last about a day).

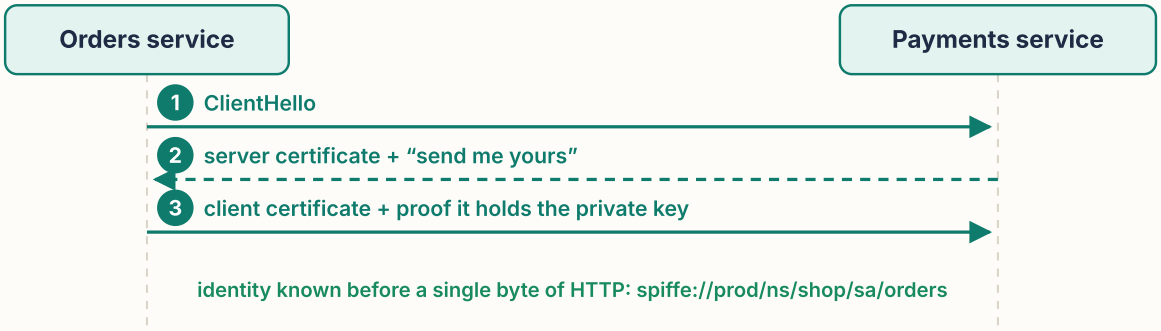


Figure 3. Mutual TLS. The caller's identity comes from its certificate, here a SPIFFE ID naming the service.

Method	Identifies	Secret sent?	Revocation	Best for
API key	The calling app	Yes, every request	Delete the key	Simple public APIs, usage tracking
Session cookie	A user in a browser	Session ID, every request	Instant (delete session)	Web apps
JWT	User or service + claims	Token, every request	Hard: wait for expiry	Many services verifying locally
Opaque token	User or service	Token, every request	Instant	When revocation matters
HMAC signing	Key holder + body integrity	**No**, only a signature	Rotate the secret	Webhooks, partner APIs
mTLS	The calling service	**No**, private key stays local	Short-lived certs	Internal service-to-service

COMMON MISTAKES

- Shipping an API key in a mobile app or JavaScript bundle.
- Long-lived JWTs (days or weeks) with no way to revoke them.
- Verifying a webhook signature over parsed and re-serialized JSON, or comparing signatures with `==`.
- Signing requests without a timestamp, so a captured request can be replayed forever.

6 Key terms

Bearer secret: whoever holds it can use it

Client credentials: OAuth 2 flow for services

Constant-time compare: takes the same time on any mismatch

JWT: signed, self-contained token

HMAC: keyed hash proving sender + integrity

mTLS: both sides present certificates

Opaque token: random reference, looked up

Replay attack: resending a captured request

Service mesh: automates mTLS between services

7 Interview questions

Q1. How would you authenticate server-to-server calls? BASIC

Mutual TLS, or OAuth 2 client credentials with short-lived tokens, or HMAC-signed requests, rather than long-lived static API keys alone. Inside a mesh, mTLS is usually automatic.

Q2. Why sign requests with HMAC? BASIC

It proves both the sender and the integrity of the body without ever sending the secret. With a timestamp in the signed string, and a window like 5 minutes, it also prevents replay attacks.

Q3. Your mobile app needs to call your API. Can it use an API key? SCENARIO

No: any key in the app can be extracted. Have the user log in and use OAuth 2 (authorization code with PKCE) to get short-lived access tokens tied to that user. Use API keys only for server-side callers.

Q4. How do you verify a payment provider's webhook? SCENARIO

Read the raw body before parsing, compute HMAC-SHA256 over timestamp + body with the webhook secret, compare in constant time, reject timestamps older than 5 minutes, then process idempotently by event ID.

Q5. A customer's API key leaked on GitHub. What should your design already support? SCENARIO

Hashed storage (the leak isn't in your DB), instant revocation, two active keys per customer so they can rotate without downtime, per-key scopes to limit damage, and usage logs per key to see what the attacker did.

Q6. Why compare HMAC signatures in constant time? TRICKY

A normal comparison returns as soon as one byte differs, so its timing reveals how many leading bytes were right. An attacker can guess the signature byte by byte. `hmac.compare_digest` takes the same time regardless.

Q7. With mTLS everywhere, do you still need authorization? TRICKY

Yes. mTLS proves which service is calling; it doesn't say what that service may do, or on behalf of which user. You still need policies (orders may call payments' charge endpoint, not refunds) and user context, such as a forwarded token.

Next: SD-C-016 · API Authorization & BOLA backendroadmap.com